



IIC2343 - Arquitectura de Computadores (I/2026)

Guía de ejercicios: RISC-V (Subrutinas Y Convención)

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas
(mario.denzel@estudiante.uc.cl), José Mendoza (jfmendoza@uc.cl)

Pregunta 1: Ejercicio programacion de subrutinas

Implemente en lenguaje ensamblador RISC-V dos subrutinas. Para este ejercicio, no es necesario seguir la convención de llamadas estrictamente, solo tienes que cuidar de respaldar **ra** para que no se pierda entre llamadas de subrutinas.

1. **Subrutina multiplicar_por_dos:** Recibe un número entero en **a0**, lo duplica (calcula $a0 \cdot 2$) y retorna el resultado en **a0**.
2. **Subrutina calcular_perimetro:** Recibe el ancho de un rectángulo en **a0** y el alto en **a1**. Debe calcular el perímetro total ($2 \cdot ancho + 2 \cdot alto$) llamando a la subrutina **multiplicar_por_dos** para cada lado, y retornar el resultado final en **a0**.

Se da como base el siguiente código:

```
.data
    ancho:    .word    5
    alto:     .word    3
.text
main:
    la    t0, ancho
    lw    a0, 0(t0)      # a0 = ancho (5)
    la    t0, alto
    lw    a1, 0(t0)      # a1 = alto (3)

    jal   ra, calcular_perimetro # Llamada a la función principal

    addi  a7, zero, 10     # Terminar ejecución
    ecall

multiplicar_por_dos:
    # Completa el codigo aqui
    jalr  zero, 0(ra)

calcular_perimetro:
    # Completa el codigo aqui
    jalr  zero, 0(ra)
```

Solución:

```
multiplicar_por_dos:
    add  a0, a0, a0          # a0 = a0 * 2
    jalr zero, 0(ra)

calcular_perimetro:
    addi sp, sp, -4         # Reservar espacio en stack
    sw   ra, 0(sp)          # Guardar ra original

    addi t0, a1, 0          # t0 = ancho
    jal  ra, multiplicar_por_dos # a0 = 2 * ancho

    addi t1, a0, 0          # t1 = 2 * ancho
    addi a0, t0, 0          # a0 = ancho
    jal  ra, multiplicar_por_dos # a0 = 2 * ancho

    add  a0, t1, a0          # a0 = (2 * ancho) + (2 * ancho)

    lw   ra, 0(sp)          # Restaurar ra original
    addi sp, sp, 4          # Liberar stack
    jalr zero, 0(ra)
```

Pregunta 2: Explique el código [AP-S2-2025'2]

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_n_m`:

```
.data
n:  .word 37
m:  .word 5
r:  .word 0
.text
main:
    addi sp, sp, -16
    sw ra, 12(sp)
    la t0, n
    lw a0, 0(t0)
    la t1, m
    lw a1, 0(t1)
    jal ra, func_n_m
    la t2, r
    sw a0, 0(t2)
    lw ra, 12(sp)
    addi sp, sp, 16
    addi a7, zero, 10
    ecall

func_n_m:
    beq a0, zero, ccc
    addi sp, sp, -16
    sw ra, 12(sp)
    sw a0, 8(sp)
    addi a0, a0, -1
    jal ra, func_n_m
    lw t0, 8(sp)
    lw ra, 12(sp)
    addi sp, sp, 16
    add a0, a0, a1
    jalr zero, 0(ra)

ccc:
    addi a0, zero, 0
    jalr zero, 0(ra)
```

Este fragmento representa el cómputo de una función $f(n, m)$. A partir de este:

1. **(1.5 ptos.)** Indique, con argumentos y en términos de n y m , lo que retorna la función $f(n, m)$.
Por ejemplo, $f(n, m) = n + m$. Se otorgan 0.75 ptos. por la correctitud de la descripción del retorno y 0.75 ptos. por justificación.
2. **(1.5 ptos.)** Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V.
Se otorgan 0.75 ptos. si indica de forma correcta si se respeta o no la convención y 0.75 ptos. por entregar una justificación válida respecto a su respuesta.

Solución:

1. La función implementa la multiplicación mediante sumas sucesivas, por lo que:

$$f(n, m) = n \times m$$

En el código, `a0` contiene el argumento n y actúa como contador decreciente en cada llamado recursivo. El caso base ocurre cuando `a0` = 0, retornando 0. En cada retorno desde la recursión, se suma `a1` (que contiene m) al valor acumulado en `a0`, de modo que se realizan n sumas del valor m . El resultado final ($n \times m$) queda almacenado en `a0`, y luego se guarda en `r`.

2. El fragmento anterior **NO** respeta la convención de llamadas de RISC-V. Aunque el código ejecuta correctamente la multiplicación y maneja el stack sin errores de ejecución, desde el punto de vista de la convención formal no cumple con las reglas establecidas. En particular:

- Los registros `a0` y `a1` son *caller-saved*. En el programa principal (`main`) se realiza el llamado a `func_n_m` sin respaldar previamente ninguno de estos registros, a pesar de que ambos pueden ser modificados por la subrutina.
- Dentro de la subrutina `func_n_m`, el registro `a1` tampoco es respaldado antes del llamado recursivo, incumpliendo nuevamente el requisito de *caller-saved*, ya que la función se llama a sí misma y puede alterar sus propios argumentos.
- El registro `t0`, que es de tipo *caller-saved*, se utiliza dentro de la función después del retorno recursivo sin haber sido respaldado antes del llamado, lo que también infringe la convención.

Pregunta 3: Ejercicio de Programación y Convención de Llamadas

Considere un sistema embebido que recibe bloques de datos de un sensor. Se le solicita implementar en lenguaje ensamblador RISC-V dos subrutinas que permitan filtrar y promediar estos datos.

1. **Subrutina `filtrar_valor`:** Recibe un entero por valor en el registro `a0`. Si el entero es negativo, lo transforma en 0. Si es positivo, lo mantiene igual. Retorna el valor filtrado en `a0`.
2. **Subrutina `procesar_buffer`:** Recibe en `a0` la dirección de inicio de un arreglo de enteros de 32 bits y en `a1` el tamaño N del arreglo. Esta función debe recorrer el arreglo, aplicar la función `filtrar_valor` a cada elemento, modificar el arreglo original en memoria con los nuevos valores filtrados, y finalmente **retornar la suma** de todos los valores ya filtrados en `a0`.

Desarrolle el código en ensamblador RISC-V para ambas subrutinas, respetando la convención de llamados para el respaldo de registros en el *stack*. Se da como base el siguiente código:

```
.data
# Arreglo de prueba con valores positivos y negativos
buffer:    .word 5, -3, 12, 0, -8, 7
tamaño:    .word 6

.text

main:
    # Cargar dirección del buffer y el tamaño
    la a0, buffer          # Carga dirección base de prueba
    la t0, tamaño
    lw a1, 0(t0)           # a1 = 6

    jal ra, procesar_buffer # Llamada a la función principal del ejercicio

    # Al volver, a0 contiene la suma total. dirección
    # Terminar ejecución
    addi a7, zero, 10      # Código de salida
    ecall

filtrar_valor:
    # Completa el código aquí
    jalr zero, 0(ra)

procesar_buffer:
    # Completa el código aquí
    jalr zero, 0(ra)
```

Solución:

```
filtrar_valor:
    blt a0, zero, es_negativo
    jalr zero, 0(ra)

    es_negativo:
        addi a0, zero, 0
        jalr zero, 0(ra)

procesar_buffer:
    addi sp, sp, -24
    sw ra, 20(sp)          # Guardar dirección de retorno original
    sw s0, 16(sp)          # s0 = dirección del elemento actual
    sw s1, 12(sp)          # s1 = contador N (elementos restantes)
    sw s2, 8(sp)           # s2 = acumulador de la suma

    addi s0, a0, 0          # s0 = dirección del primer elemento
    addi s1, a1, 0          # s1 = N
    addi s2, zero, 0        # s2 = suma inicializada en 0

    loop:
        beq s1, zero, fin   # Si N == 0, salir del ciclo

        lw a0, 0(s0)        # Cargar elemento actual de la memoria

        jal ra, filtrar_valor # Llamar a la subrutina, resultado en a0

        sw a0, 0(s0)        # Guardar el valor filtrado en la memoria

        add s2, s2, a0       # Acumular en la suma

        addi s0, s0, 4       # Avanzar al siguiente entero (4 bytes)
        addi s1, s1, -1      # Decrementar el contador N

        jal zero, loop

    fin:
        addi a0, s2, 0       # Colocar la suma total en a0

        lw ra, 20(sp)
        lw s0, 16(sp)
        lw s1, 12(sp)
        lw s2, 8(sp)
        addi sp, sp, 24

        jalr zero, 0(ra)
```